

# DEVFLOW: Interview Preparation Guide

A Codebase-Grounded Study Document & Architectural Review | Stack: MERN + Grounded AI

## 1. Executive Summary & Core Problem

**What is DevFlow?** DevFlow is a full-stack developer workspace built to overcome **algorithmic memory decay** during software engineering interview preparation. Engineers frequently solve hundreds of LeetCode or DSA problems, but forget the underlying patterns weeks later due to Hermann Ebbinghaus's forgetting curve. DevFlow automates review intervals at **Day 1, 3, 7, 15, and 30**, while centralizing technical notes, streak analytics, and grounded AI interview intelligence.

## 2. System Architecture & Component Trace

DevFlow operates as a decoupled single-page application communicating over a REST API with Bearer token authentication.

- **Client:** React 19 single-page application built with Vite and Tailwind CSS v4. Centralized api.js handles token management.
- **Routing & Auth:** React Router v7 with <ProtectedRoute> checking JWT validity against /api/users/me on mount.
- **Server:** Node.js and Express 5 REST API organized into Routes, Controllers, Services, and Models.
- **Security:** authMiddleware.js verifies cryptographic JWT signatures and attaches req.user = { userId } for strict query scoping.
- **Database:** MongoDB document database with Mongoose schemas, compound indexes, and aggregation pipelines.
- **AI Pipeline:** Proxied requests with 15s timeout, regex JSON extraction, schema normalization, and an offline deterministic heuristic fallback.

## 3. Production Tech Stack & Rationale

| Technology          | Role in DevFlow     | Why Chosen                             | Alternative      |
|---------------------|---------------------|----------------------------------------|------------------|
| React 19 + Vite     | Client UI & forms   | Lightweight virtual DOM, fast HMR      | Vue / Angular    |
| Tailwind CSS v4     | Design system       | Atomic classes, custom design tokens   | CSS Modules      |
| Node.js + Express 5 | REST API Server     | Non-blocking event loop ideal for I/O  | Spring Boot / Go |
| MongoDB + Mongoose  | Persistence         | Flexible document model for notes & AI | PostgreSQL       |
| JWT Authentication  | Session state       | Stateless verification; no DB lookups  | Redis Sessions   |
| AI Pipeline         | Career intelligence | OpenAI/Gemini with Heuristic fallback  | Self-hosted LLMs |

## 4. Core Feature Implementations

### Spaced Repetition Engine (Day 1, 3, 7, 15, 30)

When a problem is marked 'Solved' in `problemService.js`, `revisionService.scheduleProblemRevisions()` executes an `insertMany()` of 5 Revision documents with scheduled dates set to `baseDate + [1, 3, 7, 15, 30]` days. Marking a revision complete increments the parent problem's revision count and automatically advances `nextRevisionDate`.

### Database Compound Indexing

In `models/Problem.js`, compound indexes such as `{ user: 1, status: 1 }` and `{ user: 1, topic: 1 }` ensure that every query scoped by user ID uses fast index scans (IXSCAN) rather than scanning the entire collection (COLLSCAN).

### Security & IDOR Prevention

DevFlow strictly prevents Insecure Direct Object References (IDOR). The backend never trusts client-supplied user IDs; all database mutations are scoped using `{ _id: req.params.id, user: req.user.userId }` derived from the verified JWT signature.

## 5. Top 10 Must-Know Interview Questions & Answers

### 1. Tell me about your project.

**Answer:** DevFlow is a full-stack developer workspace designed to stop algorithmic memory decay during interview prep. It pairs a LeetCode tracker with an automated 5-stage spaced repetition schedule (Day 1, 3, 7, 15, 30), technical markdown notes, streak analytics, and grounded AI tools for resume ATS analysis and dynamic interview question generation.

### 2. How does authentication work end-to-end?

**Answer:** Users register with email and password, which is hashed with `bcrypt` (10 rounds). An OTP code verifies email ownership. The server issues a signed JWT (1d expiry). The client stores the token in `localStorage` and transmits it in the `Authorization: Bearer` header. The backend `authMiddleware` validates the token and attaches `req.user = { userId }`.

### 3. How do you prevent users from tampering with other users' data?

**Answer:** We never accept `userId` from the frontend body or URL. We extract `userId` exclusively from the verified JWT. Every database read, update, or delete strictly queries `{ _id: resourceId, user: req.user.userId }`. If another user's ID is requested, MongoDB returns null, returning a 404.

### 4. How does the spaced repetition algorithm work in code?

**Answer:** When a problem status is set to 'Solved', `problemService` invokes `revisionService.scheduleProblemRevisions()`. It maps over interval days `[1, 3, 7, 15, 30]` and executes an `insertMany` of five Revision records. Completing a revision increments `revisionCount` and advances `nextRevisionDate`.

### 5. How do you guarantee AI reliability and prevent invalid JSON crashes?

**Answer:** All AI calls go through our backend with 15s timeouts and retries. We use `extractJson()` to strip markdown fences (````json`) and parse root JSON brackets. We then validate and clamp scores between 0-100. If external APIs fail or keys are absent, our deterministic `heuristicEngine.js` runs an SDE taxonomy analysis, ensuring zero downtime.

### 6. How do you ensure the AI generates fresh interview questions every time?

**Answer:** We bypass caching (`skipCache: true`), set temperature to 0.8 for diverse token generation, and pass `previousQuestions` in the request body. The system prompt explicitly commands the model not to repeat or rephrase any previous questions.

### 7. Why choose MongoDB over PostgreSQL for DevFlow?

**Answer:** MongoDB's flexible document model natively stores polymorphic technical notes, code snippets, and nested AI-generated ATS feedback without rigid table migrations. When ACID transactions across 20 tables are not required, MongoDB provides faster iterative development.

### 8. What is the difference between client-side and server-side filtering in DevFlow?

**Answer:** Early tasks used client-side filtering on the fetched array. Our core problems feature uses 100% server-side filtering: status, difficulty, topic, and regex search parameters are passed directly to MongoDB queries using indexes and skip/limit pagination, minimizing network payload.

### 9. How are user streaks calculated?

**Answer:** In analyticsService.js, we query all distinct solved dates, format them as unique YYYY-MM-DD strings in a sorted set, and compute consecutive calendar day differences. We also verify that the streak extends to today or yesterday so broken streaks reset to zero.

### 10. What would you improve before deploying to high-traffic production?

**Answer:** I would migrate JWT storage from localStorage to HttpOnly Secure cookies to eliminate XSS risks, implement BullMQ background job queues for revision notifications, and add a Redis cache layer for user profile context and analytics summaries.

## 6. Fast Review Cheat Sheet

- **Core Pitch:** DevFlow stops algorithmic memory decay using automated spaced repetition.
- **5 Intervals:** Day 1 (+24h), Day 3 (+72h), Day 7 (+1w), Day 15 (+2w), Day 30 (+1m).
- **IDOR Defense:** Database queries enforce { \_id: id, user: req.user.userId }.
- **Compound Indexes:** { user: 1, status: 1 } enables fast index scans (IXSCAN).
- **AI Pipeline:** Backend proxy, 15s timeout, JSON sanitization, and deterministic heuristic fallback.
- **Dynamic Questions:** Temperature 0.8, zero caching, and anti-duplication exclusion lists.